

Physics-Informed neural networks for the time-dependent Schrödinger equation

Afthash Puzhakkal¹

¹*The University of Adelaide, North Terrace Campus, Adelaide, 5000, South Australia, Australia*

Abstract: Solving the time-dependent Schrödinger equation (TDSE) for high-dimensional systems can be computationally expensive using numerical methods such as Crank-Nicholson. Physics-informed Neural Networks (PINNs) offer a promising alternative by using deep learning techniques to find the solution by enforcing physical laws through loss functions. In this research project, my aim is to develop a PINN model to solve the TDSE. The study began with a single-electron system in a one-dimensional harmonic potential, then I approached a more complex problem, an electron in a moving quantum dot. These models were evaluated against known analytical or numerical solutions based on the probability density plot and wavefunction normalization plot. Later in the research, advanced training strategies such as causal training and normalization enforcement were explored, which improved the performance of the vanilla moving quantum dot model. Finally, I trained the model on a two-dimensional setup, demonstrating that the PINNs are capable of handling higher dimensions. In general, this research project explored a computationally efficient and scalable alternative to analytical and numerical methods for solving complex quantum problems.

CONTENTS

A. Additional Plots

10

I. Introduction	1
II. Literature review	2
III. Methodology	2
A. Physical modeling	2
1. One-dimensional static quantum dot	2
2. One-dimensional moving quantum dot	2
3. One-dimensional dynamic-window	3
4. Two-dimensional dynamic-window	3
B. PINN Model Architecture	3
C. Loss Function	3
D. Data Sampling and Training	4
E. Training Strategies	4
1. Causal Training	4
2. Normalization Enforcement	4
F. Model Evaluation	4
IV. Ethics	5
V. Results	5
A. 1-D static quantum dot	5
B. 1-D moving quantum dot	5
1. Vanilla Model	5
2. Effect of Causality	6
3. Effect of Normalization Enforcement	7
C. 1-D dynamic-window	7
D. 2-D dynamic-window	7
E. Summary of results	8
VI. Discussion	8
VII. Conclusion	9
Abbreviations	9
Acknowledgments	9
References	9

I. INTRODUCTION

The Schrödinger equation describes how the wavefunction of a quantum system evolves over time and space. Traditionally, the Schrödinger equation can be solved using analytical methods for simple systems and numerical methods for complex systems. Numerical methods such as Crank-Nicholson, while robust, become computationally expensive for higher-dimensional problems [1]. This creates the need for an efficient solver to solve higher-dimensional time-dependent quantum problems.

Physics-informed neural networks (PINNs), introduced by Raissi *et al.* [2], offer one such solver. They are deep learning models that learn to solve partial differential equations (PDEs) like the Schrödinger equation by including the physical laws directly into the model's loss function. The objective of this research is to understand whether physics-informed neural networks can solve the time-dependent Schrödinger equation accurately and efficiently.

I started the study by designing and creating a PINN architecture to solve the Schrödinger equation for a single electron trapped inside a one-dimensional harmonic potential. Later, I solved the TDSE for an electron inside a moving quantum dot. I verified the effectiveness of my models using the results from the Crank-Nicholson method. Later, to improve the effectiveness of the model, advanced training strategies such as causal training, where I ensure that the model does not violate causality during training [3], and normalization enforcement, where I explicitly force wavefunction normalization at selected times to stay at 1 during training, were implemented. Finally, I trained the dynamic part of the moving quantum dot system separately, both in 1-D and 2-D, to explore the movement of the quantum dot more closely.

II. LITERATURE REVIEW

Solving the Schrödinger equation has always been a central part of quantum mechanics. For most real-world systems, researchers have to rely on numerical methods such as the Crank-Nicholson method to solve the TDSE, which is accurate and reliable but becomes slow and expensive when solving time-dependent problems in higher dimensions.

The study by Gardin *et al.* [4] explored the behavior of an electron inside a moving quantum dot and solved the 2-D TDSE using the Crank-Nicholson method. Their work highlighted how the speed of a quantum dot affects the wavefunction's probability transitions. This study provides a strong physical background and reference data for applying PINNs to the moving quantum dot problem, making it relevant for this project.

In a paper by Raissi *et al.* [2], PINNs were introduced as a method to solve forward and inverse problems involving PDEs. Unlike traditional solvers, which follow a grid-based approach, PINNs follow a mesh-free approach and learn by minimizing the loss function, which involves the physical law described by the PDE. This makes PINNs a better choice where high-resolution or adaptive domains are required. Their work showed successful results in equations such as Burger's equation and the Navier-Stokes equation. This paper can act as a foundation for applying PINN to quantum mechanics.

Based on this, Shah *et al.* [5] applied PINNs to solve the TDSE for a 1-D harmonic oscillator. Their work showed that PINNs could predict the evolution of wavefunctions quite accurately, and they used known analytical solutions as a reference. However, they also found that performance dropped for longer time ranges and higher-energy states. To overcome this, they mentioned a method called causal training, which improves the overall performance of the model by training in a time-ordered manner. A further elaborative study about causality can be found in a paper by Wang *et al.* [3]. In that study, they proved that respecting the natural flow of time during training will help the model to learn more realistically.

Although PINNs have their strengths, they also have some limitations. Grossmann *et al.* [1] performed a detailed comparison between PINN and finite element method (FEM). They found that while PINNs are flexible, FEM is still more accurate and stable in many situations. This is especially true for large-scale problems or systems with stiff equations. Their work is a clear reminder that PINNs need improvements before they replace traditional solvers.

In summary, the literature shows that PINNs are a powerful new approach for solving quantum problems. They offer flexibility and grid-free modeling but still face accuracy and stability issues over longer time evolution. With additional techniques like causal training and normalization enforcement, this issue can be solved to an extent, and therefore PINNs can be considered as an alternative or complement to traditional solvers.

III. METHODOLOGY

A. Physical modeling

The Schrödinger equation for a 1-D system, which describes how the wavefunction $\psi(x, t)$ evolves over time, can be expressed as:

$$i\hbar \frac{\partial \psi(x, t)}{\partial t} = -\frac{\hbar^2}{2m} \frac{\partial^2 \psi(x, t)}{\partial x^2} + V(x, t)\psi(x, t) \quad (1)$$

Here, $\hbar$ is the reduced Planck's constant, m is the particle mass, and $V(x, t)$ is the potential energy function.

1. One-dimensional static quantum dot

My study began by recreating the baseline model from the paper by Shah *et al.* [5], where they solve the Schrödinger equation for an electron trapped in a 1-D harmonic potential. In this paper, they adopted Hartree atomic units ($\hbar = m_e = 1$), so energies are measured in Hartree and lengths in Bohr radii. The initial condition is given as a superposition of the ground and the first excited state.

The Schrödinger equation for this system will then be:

$$i\frac{\partial \psi(x, t)}{\partial t} = -\frac{\partial^2 \psi(x, t)}{\partial x^2} + V(x)\psi(x, t) \quad (2)$$

where,

$$V(x) = \frac{1}{2}m\omega^2x^2 = \frac{1}{2}\omega^2x^2 \quad (3)$$

where ω determines the curvature of the potential. This corresponds to what we refer to as the baseline model in what follows.

2. One-dimensional moving quantum dot

Later, I extend my problem to a system where an electron is in a time-dependent moving quantum dot. From this problem onwards, I will be using nanometer (nm), picosecond (ps), and milli electron volt (meV) units. The new system's potential energy is:

$$V(x, t) = \frac{1}{2}m\omega^2(x - x_{qd}(t))^2 \quad (4)$$

In this context, $x_{qd}(t)$ denotes the position of the moving quantum dot at time t , as follows:

$$x_{qd} = \begin{cases} x_0, & t \leq t_0 \\ x_0 + v_{qd}(t - t_1), & t_0 < t \leq t_1 \\ x_1, & t \geq t_2 \end{cases} \quad (5)$$

where,

$$t_2 = t_1 + \frac{x_1 - x_0}{v_{qd}} \quad (6)$$

and,

- x_0 : Initial position of quantum dot
- x_1 : Final position of quantum dot
- t_0 : Start time of the simulation
- t_1 : Start time of quantum dot movement
- t_2 : Stop time of quantum dot movement
- v_{qd} : Velocity of the quantum dot movement

This dynamic scenario is inspired by Gardin *et al.* [4], who explored non-adiabatic transitions in moving quantum dots.

3. One-dimensional dynamic-window

Later, I trained a new model to get a closer look at the dynamic part of the moving quantum dot system, which is the window between the time intervals t_1 and t_2 where the quantum dot actually moves in space. I will refer to this model as the dynamic-window model.

4. Two-dimensional dynamic-window

Finally, I created a 2-D version of the dynamic-window model, designed in a way such that the quantum dot moves through the (x, y) plane diagonally. The potential energy of the new system will be:

$$V(x, y, t) = \frac{1}{2}m\omega^2(x - x_0)^2 + \frac{1}{2}m\omega^2(y - y_0)^2 \quad (7)$$

where (x_0, y_0) is the position of the quantum dot in the (x, y) plane.

B. PINN Model Architecture

The PINN model takes position x and time t as inputs and gives the real and imaginary parts of the wavefunction ψ as output. I used a fully connected feedforward neural network with 6 hidden layers with 512 neurons each with an activation function tanh for the baseline PINN model and 12 hidden layers with 512 neurons each for the moving quantum dot model with the tanh activation function for the first layer and the SiLU activation function for the rest of the layers. The PINN learns by minimizing a loss function that enforces the governing PDE, initial condition, and boundary conditions.

C. Loss Function

As mentioned in earlier sections, a PINN model solves a problem by learning the physical laws of the system represented through a loss function. The loss function

includes not only the PDE itself but also the initial and boundary conditions. The model will then be trained to minimize this loss function, which will result in learning the PDE and other conditions.

The total loss L_{total} can be defined as:

$$L_{total} = \lambda_{pde}L_{pde} + \lambda_{ic}L_{ic} + \lambda_{bc}L_{bc} \quad (8)$$

where,

- L_{pde} : Physics loss (PDE loss), handle the overall learning of the system's PDE
- L_{ic} : Initial condition loss, handle learning the wavefunction at $t = 0$
- L_{bc} : Boundary condition loss, handle learning the wavefunction at the spatial boundaries

Here $\lambda_{pde}, \lambda_{ic}, \lambda_{bc}$ are the regularization parameters or weights of the respective individual losses, through which we can control how much importance the model should give to learning each loss during training.

The residual of the Schrodinger equation, R_{pde} , can be written as:

$$R_{pde} = i\hbar \frac{\partial \psi(x, t)}{\partial t} + \frac{\hbar^2}{2m} \frac{\partial^2 \psi(x, t)}{\partial x^2} - V(x, t)\psi(x, t) \quad (9)$$

and during training, model will learn the solution by minimizing this residual.

Since the wavefunction $\psi(x, t)$ is a complex number and PINNs can only handle real-valued variables, we need to rewrite it as:

$$\psi(x, t) = u + iv \quad (10)$$

we will now substitute this in equation 9 and separate the real and imaginary part of the residual. Which will give us:

$$R_{pde}^{real} = -\hbar \frac{\partial v}{\partial t} + \frac{\hbar^2}{2m} \frac{\partial^2 u}{\partial x^2} - V(x, t)u \quad (11)$$

and,

$$R_{pde}^{img} = i(\hbar \frac{\partial u}{\partial t} + \frac{\hbar^2}{2m} \frac{\partial^2 v}{\partial x^2} - V(x, t)v) \quad (12)$$

and using these values, we can define L_{pde} as:

$$L_{pde} = \frac{1}{N_{colloc}} \sum^{N_{colloc}} (R_{pde}^{real})^2 + (R_{pde}^{img})^2 \quad (13)$$

where N_{colloc} is the number of collocation points, or the number of samples used to calculate physics loss.

Similarly, we can enforce an analytical initial condition to the model's predicted u, v when $t = 0$. That will be the initial condition loss, or L_{ic} . Likewise, for the boundary condition, if we are using Dirichlet boundary conditions, both u and v should be 0 at spatial boundaries, and we can ensure this using boundary condition loss, or L_{bc} .

D. Data Sampling and Training

For the baseline model, I used collocation points $(x, t) \in [x_{min}, x_{max}] \times [t_{min}, t_{max}]$ following random uniform sampling, and for the rest of the models, the collocation points follow a random normal distribution, centered around the position of the quantum dot at a given time. Initial condition points are selected as uniform x points when $t = 0$, while boundary points are fixed at spatial domain edges.

The networks were trained for 250,000 epochs using the Adam optimizer with a learning rate $\gamma = 0.001$ that undergoes an exponential decay. I used the PyTorch framework in Python for building and training the models. Computations were performed on an NVIDIA A100 GPU.

E. Training Strategies

Everything up to now is essentially what we call the vanilla model. Now we will discuss the two additional training strategies extending the vanilla moving quantum dot model.

1. Causal Training

Causal training makes sure that the model's learning happens in a time-ordered manner. In other words, physics loss at later segments should depend on the same from earlier segments. Training the model this way will satisfy the principle of causality, which is inherent to quantum systems. To ensure causality, the physics loss is modified following Wang *et al.* [3] as:

$$L_{pde} = \frac{1}{N_{seg}} \sum_{i=1}^{N_{seg}} \left(\sum_{k=1}^{i-1} L_k \right) + L_i \quad (14)$$

where,

- N_{seg} is the number of segments used
- L_i and L_k are L_{pde} given by eq. (13) for segment i and k respectively

For the rest of this report, the model in which causality is enforced will be referred to as the causal model.

2. Normalization Enforcement

When solving Schrödinger equation, normalizing the wavefunction during the simulation ensures that the wavefunction follows a valid probability distribution. In other words, the probability of finding a particle anywhere in the spatial domain should stay at 1 all the time.

Mathematically, this can be represented as,

$$\int |\psi(x, t)|^2 dx = 1, \quad (15)$$

But when training a model using PINNs, we don't explicitly normalize the wavefunction; instead, the final wavefunction is expected to stay normalized as a result of the model learning the system correctly.

As part of improving my vanilla quantum dot model, I decided to manually enforce the wavefunction to stay normalized during training with the help of an additional term to the loss function. During training, I used the Monte Carlo method to find an approximation of the integration represented in equation 15. The Monte Carlo approximation at time t can be expressed as:

$$R_{norm} = (x_{max} - x_{min}) \frac{\sum_{i=0}^{N_{mc}} \psi(x_i, t)}{N_{mc}} \quad (16)$$

where R_{norm} will be the normalization residual and N_{mc} is the number of wavefunction samples we used to find the approximation. Building on top of this, the normalization loss, or L_n , can be defined as:

$$L_n = \frac{1}{N_{norm}} \sum (R_{norm} - 1)^2 \quad (17)$$

so that minimizing L_n will bring the value of R_{norm} closer to 1. Here, N_{norm} is the number of equidistant time points we selected to find the normalization loss. If we included L_n in the total loss (equation 8), the new total loss will be,

$$L_{total} = \lambda_{pde} L_{pde} + \lambda_{ic} L_{ic} + \lambda_{bc} L_{bc} + \lambda_n L_n \quad (18)$$

where λ_n is the weight of normalization loss.

For the rest of this report, the model in which wavefunction normalization is enforced will be referred to as the normalization model or the norm model.

F. Model Evaluation

I used the results from an analytical equation to evaluate the baseline model and results from the Crank-Nicholson method to evaluate the moving quantum dot models. The performance was evaluated using a comparison of the probability density plots and the wavefunction normalization plots.

The first metric used for evaluation is the maximum absolute error between the probability density predicted ($\Delta\psi^2(max)$) by the PINN model and the result from the analytical/numerical method, which can be denoted by:

$$\Delta\psi^2(max) = max(|\psi|_{predicted}^2 - |\psi|_{actual}^2|) \quad (19)$$

where $|\psi|_{predicted}^2$ and $|\psi|_{real}^2$ are the probability densities of the predicted result and the analytical/numerical result, respectively.

$\Delta\psi^2$ (max)	Wavefunction Normalization		Training Time (minutes)
	Mean	Standard deviation	
0.00940	0.99966	0.00020	71

TABLE I. Performance metrics of the Baseline model

The other two metrics are the mean and standard deviation of the wavefunction normalization. This will help to determine how much the predicted solution deviates from the physically accurate solution.

IV. ETHICS

This research project does not involve any human participants, personal data, or sensitive information. All simulations and datasets used in the study were generated synthetically using mathematical models and do not pose any ethical risks.

The main ethical consideration in this project is ensuring the integrity and reproducibility of scientific research. To address this, standard practices such as citing all references, using publicly available methods (such as Crank-Nicholson and PINNs), and documenting the experimental setup were followed carefully. No pretrained models were used. All neural network training was done from scratch using open-source libraries like PyTorch.

The computational resources used ensured that the research was conducted in an energy-efficient and accessible manner. Overall, this project aligns with academic standards for ethical machine learning and physics research.

V. RESULTS

A. 1-D static quantum dot

I started this project by developing a PINN model for a system where an electron is trapped in a 1-D harmonic oscillator. This baseline model was able to predict the wavefunction evolution correctly. The predicted probability density $|\psi(x,t)|^2$ (figure 1) closely matched the analytical results.

B. 1-D moving quantum dot

As a next step in the project, I trained the PINN model to solve a system where an electron is inside a time-dependent moving quantum dot. The probability density of this system's Crank-Nicholson solution is represented in figure 2.

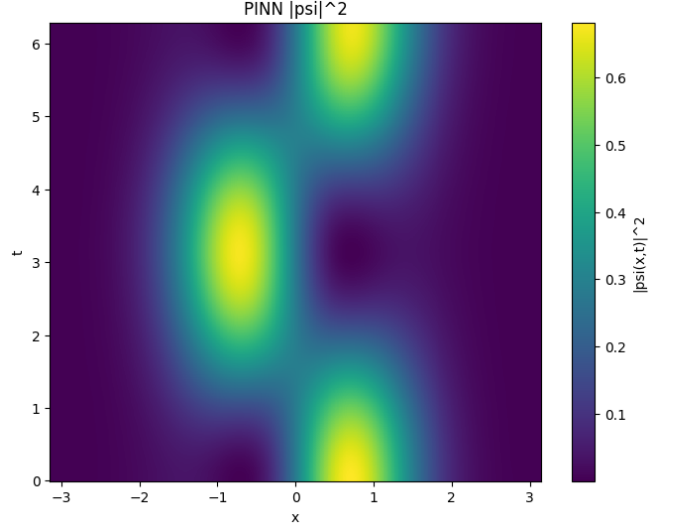


FIG. 1. Probability Density $|\psi(x,t)|^2$ of the baseline model. Here, I reproduced the results from the paper by Shah *et al.* [5], and this plot can be verified by comparing figure 1 in that paper.

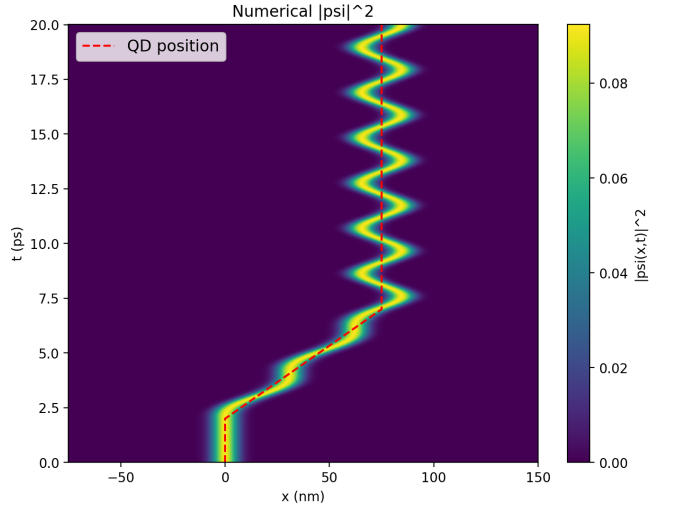


FIG. 2. Crank-Nicholson result of an electron inside a moving quantum dot system's probability density. Here, the quantum dot is moved from $x_0 = 0 \text{ nm}$ to $x_1 = 75 \text{ nm}$ with a velocity $v_{qd} = 15 \text{ nm/ps}$. This resulted in the electron inside the quantum dot undergoing non-adiabatic oscillation after the quantum dot stopped moving. In the figure, the red dotted line represents the position of the quantum dot at a given time. This result will be used to evaluate the performance of PINN models.

1. Vanilla Model

The vanilla PINN model was able to capture the overall motion of the wavefunction. However, as time progressed, the accuracy decreased. From figure 3, which is the probability density plot, you can see that the shape of

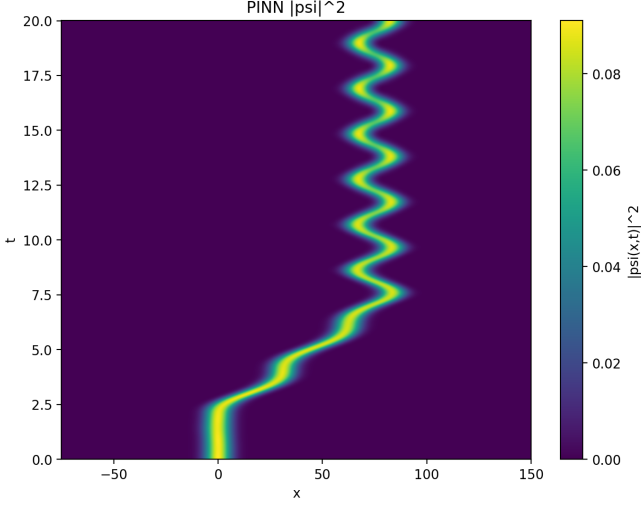


FIG. 3. Probability density $|\psi(x,t)|^2$ of the vanilla moving quantum dot model. Here, you can see that, while the quantum dot trajectory and electron's oscillation are correctly predicted, there is a clear loss of accuracy, which is visible towards the end of the simulation. This error accumulation is clearly visible in figure 4 where I plot the absolute error of probability density between the PINN prediction and Crank-Nicholson result.

Model	$\Delta\psi^2$ (max)	Wavefunction Normalization		Training Time (minutes)
		Mean	Standard deviation	
Vanilla	0.03409	0.90468	0.04191	394
Causal	0.00607	0.98813	0.00447	381
Norm	0.00982	1.00111	0.00023	492

TABLE II. Performance metrics of the moving quantum dot models. Here, you can see that both the causal and normalization enforcements clearly improved the vanilla model. The causal model has improved the most on the maximum absolute error, while the norm model improved the wavefunction normalization metrics. One thing to note here is that causal training does not increase the training time, but the normalization model does, and that is because for the normalization model, we are using an additional loss function.

quantum dot's movement and the electron's oscillations after the quantum dot stopped moving are correctly predicted. But towards the end of the simulation, a clear loss of accuracy is observed.

This can be clearly understood from the absolute error plot between the vanilla model and Crank-Nicholson results, as depicted in figure 4, and the wavefunction normalization plot in figure 7, where the vanilla model's probability reaches as low as 0.86, with a mean of 0.90468, as shown in table II.

2. Effect of Causality

To improve the performance of the vanilla model, I experimented with a causal training strategy. Through training the model in a time-ordered manner, it was pos-

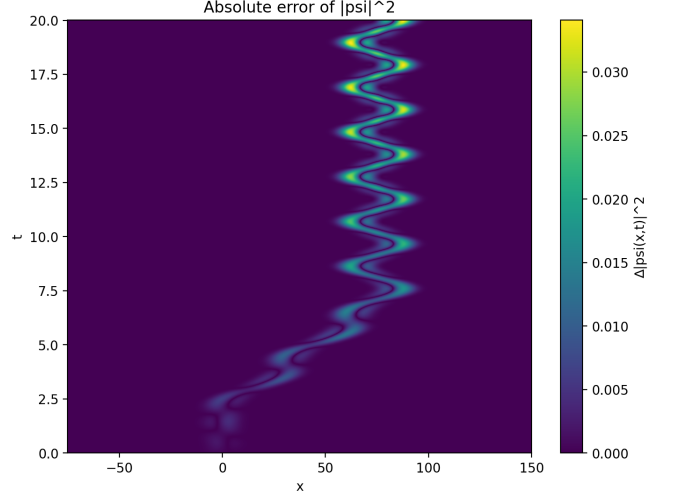


FIG. 4. Absolute error of $|\psi(x,t)|^2$ between the PINN prediction and Crank-Nicholson result. Here you can see that the error increases as the time progresses. This error accumulation is due to the PINN's inability to handle longer time evolution.

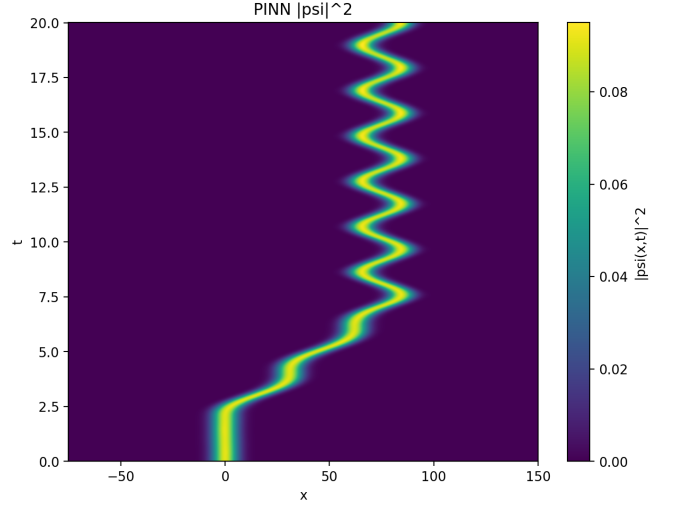


FIG. 5. Probability density $|\psi(x,t)|^2$ of the causal model. Here, the error accumulation is decreased drastically when compared to the same plot of the vanilla model (figure 3). This accuracy improvement is more obvious when you refer the wavefunction normalization plot (figure 7) where you can see how much the vanilla model improves when done causal training.

sible to satisfy the principle of causality, which resulted in the model maintaining its accuracy over time.

The improvement is clearly visible in table II or when we compare the probability density plot of the causal model (figure 5) with that of the vanilla model (figure 3).

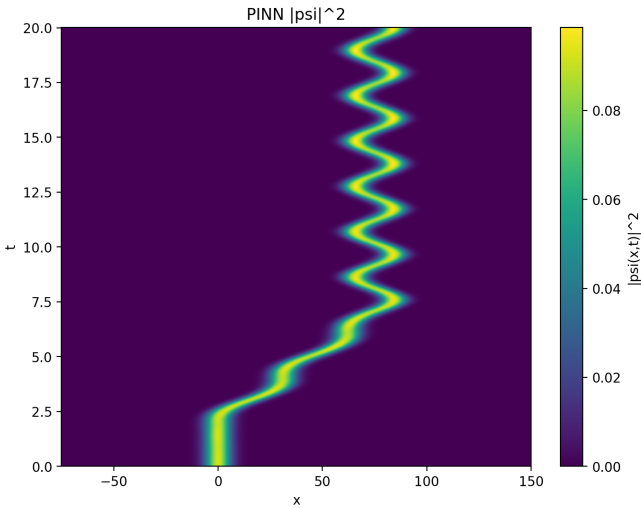


FIG. 6. Probability density $|\psi(x,t)|^2$ of the normalization model.

3. Effect of Normalization Enforcement

Another training strategy I used to improve the vanilla model is enforcing normalization during training. As you can see earlier in figure 3, the vanilla model produces non-physical results towards the end of the simulation because the wavefunction normalization deviates too much from 1.

After enforcing normalization, the normalization model consistently kept the total probability close to 1 throughout the simulation, as shown in figure 7. This resulted in the probability density plot improving (figure 6) when compared to that of the vanilla model (figure 3).

C. 1-D dynamic-window

From the results of the moving quantum dot models, I found out that the PINN struggles to learn the non-adiabatic oscillations part correctly for the vanilla model (figure 3). Even though the training strategies clearly improved the vanilla model, a question still arises: why did the vanilla model fail to learn the system properly? One of the reasons could be the fact that I am training the model for a longer time range, while PINNs have an issue with longer time evolutions [5].

A potential way to overcome this issue, other than the training strategies of course, is to train for a shorter time range. In this section, I will be showing the improvements of the model when trained only for the dynamic movement of quantum dot.

To compare its performance, if we look at the probability density plot (figure 8), we can see that unlike the vanilla model (figure 3), there are no visible loss of accuracy. This is achieved with less training time compared

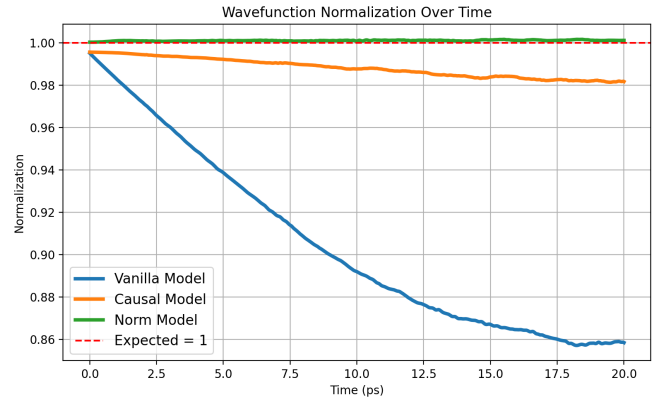


FIG. 7. Wavefunction normalization plots of the moving quantum dot models. You can see that the vanilla model deviates the most from 1 as time progresses. This issue is clearly improved by the causal model, which still shows a loss of accuracy over time; the rate of loss has been reduced by a lot when compared to the vanilla model. The norm model, where I enforce normalization, shows the effectiveness of the training strategy by staying closer to 1 all the time, without showing any loss of accuracy.

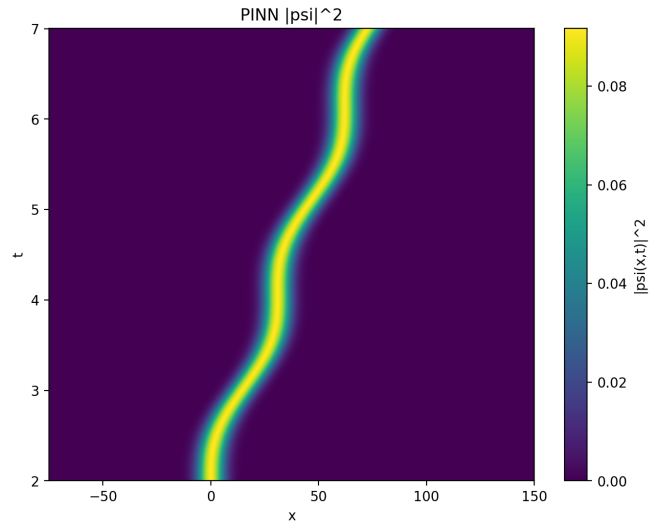


FIG. 8. Probability density $|\psi(x,t)|^2$ of the dynamic-window model.

to the vanilla model, as shown in table III.

D. 2-D dynamic-window

After noticing an increase in model accuracy with a less complex model for the Dynamic-Window model, I decided to train a model of the same system in two dimensions. Solving the TDSE for a 2-D system will give a clear idea about how both PINN and Crank-Nicholson handle the complexity of higher dimensions.

The PINN model not only predicted the wavefunction

Model	$\Delta\psi^2$ (max)	Wavefunction Normalization		Training Time (minutes)
		Mean	Standard deviation	
Vanilla*	0.01484	0.94422	0.01527	394
D-W	0.00064	0.99958	0.00084	110

TABLE III. Comparison between the vanilla model and the dynamic-window model (D-W).

Please note that here the vanilla model is only predicting the movement part of the quantum dot (t_1 to t_2), the time range in which the dynamic-window model is trained.

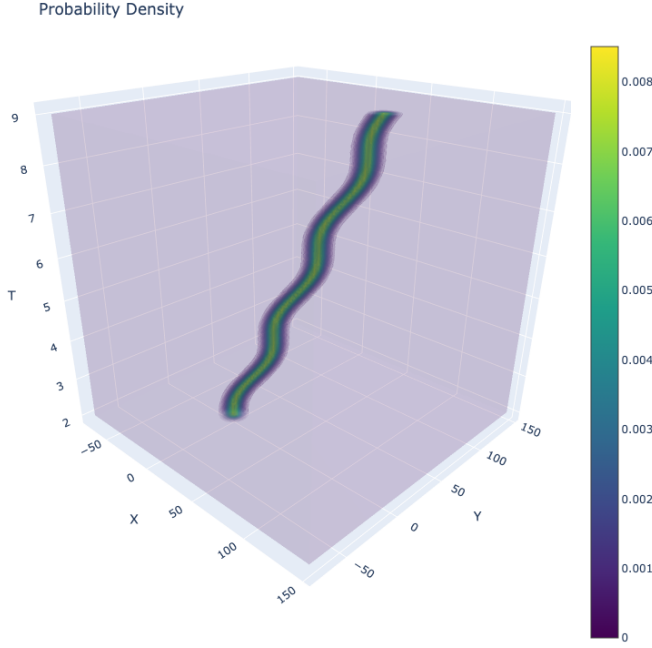


FIG. 9. Probability Density $|\psi(x, y, t)|^2$ for the 2-D dynamic-window model

for the 2-D Dynamic-Window system (figure 9), but also outperformed the Crank-Nicholson method both in training time and final output file size, as shown in table IV. The results show the robustness of PINN in handling higher-dimensional problems.

Solver	Training/simulation time (minutes)		Output size	
	1-D	2-D	1-D	2-D
PINN	110	385	5.1 MB	5.1 MB
Crank-Nicholson	0.066	443	62 MB	15 GB

TABLE IV. Training/simulation time for both the 1-D and 2-D dynamic-window models using the PINN and Crank-Nicholson method. Here, you can clearly see that PINN scales better to 2-D when compared to the Crank-Nicholson method, whose simulation time and output file size explode when scaled to 2-D.

E. Summary of results

- The baseline model successfully learned the 1-D harmonic oscillator system.
- For the vanilla moving quantum dot model, loss of accuracy was visible towards the end of the simulations.
- Causal training helped the moving quantum dot model to improve its stability.
- Similarly, normalization enforcement reduced the loss of accuracy towards the end of the time domain.
- Focusing the training only on the movement part of the quantum dot made the training more efficient and results more accurate, as shown in the results of the dynamic-window model.
- The 2-D dynamic-window model proved the PINNs' capability to handle higher dimensions better than the Crank-Nicholson method.

VI. DISCUSSION

This research project set out to explore whether PINNs can accurately solve the TDSE, especially for systems that involve a time-dependent moving potential. Based on the results, the answers are generally positive.

Starting with the 1-D harmonic oscillator, PINN showed decent performance with respect to the known analytical solution. This confirmed that the model could learn basic quantum behavior correctly [5].

However, when I tried to implement PINN to the moving quantum dot model, I faced several challenges. Firstly, I had to double the model complexity, especially to capture the electron's non-adiabatic oscillation after the quantum dot stopped moving. In addition to this, the model showed a clear loss of accuracy over longer simulation times. This is an issue PINNs face when following standard training methods [1]. The vanilla model also failed to maintain proper normalization of the wavefunction, which is critical in quantum mechanics.

To address this, I implemented two different training strategies: causal training and normalization enforcement. Causal training helped guide the model to learn in a time-ordered manner, just like in real quantum systems [3]. This improved the stability of predictions and reduced error accumulations. Similarly, enforcing normalization during training ensured that the total probability remained constant throughout the training, and this helped the model to reduce the loss of accuracy, especially towards the end of the time domain.

Another method I tried to capture the best out of PINN is to train the model for only the part where the quantum dot moves, in other words, only the dynamic-movement part. Since the training time range is reduced,

this resulted in the model improving its performance. While this approach limits generalization outside the selected time window, it is still practical for targeted quantum simulations. Finally, the same system was tested in two dimensions. PINN handled this complexity more effectively than the Crank-Nicholson method, highlighting its upper hand in handling higher-dimensional problems.

Overall, this project's results show that, while PINNs are not perfect, they can be powerful tools for solving TDSE when combined with good design choices, physics-based constraints, and training strategies. These results are important for applications in quantum computing, quantum control, and time-dependent simulations.

VII. CONCLUSION

In this research project, I used PINNs to solve the TDSE for both simple and complex quantum systems. The results showed that, while PINNs can easily learn solutions for simple problems, they might struggle with more complex problems.

By introducing training strategies such as causal training and normalization enforcement, the model was able to bring the performance of the complex systems closer to that of simple systems. This suggests that PINNs can be

a practical and flexible tool for solving time-dependent quantum problems.

Future works could look at improving training strategies, creating a more hybrid model that combines both PINN and traditional numerical methods, and more generalized models that include variables such as frequency ω , quantum dot movement speed v_{qd} , and time-dependent potential energy $V(x, t)$ as inputs.

ABBREVIATIONS

TDSE: Time-Dependent Schrödinger Equation

PINN: Physics-Informed Neural Network

PDE: Partial Differential Equation

FEM: Finite Element Method

QD: Quantum Dot

ACKNOWLEDGMENTS

This work was supported with super computing resources provided by the Phoenix HPC service at the University of Adelaide.

-
- [1] T. G. Grossmann, U. J. Komorowska, J. Latz, and C.-B. Schönlieb, Can physics-informed neural networks beat the finite element method?, *IMA Journal of Applied Mathematics* **89**, 143 (2024).
 - [2] M. Raissi, P. Perdikaris, and G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *Journal of Computational physics* **378**, 686 (2019).
 - [3] S. Wang, S. Sankaran, and P. Perdikaris, Respecting causality is all you need for training physics-informed neural networks, *arXiv preprint arXiv:2203.07404* (2022).
 - [4] A. Gardin, R. D. Monaghan, T. Whittaker, R. Rahman, and G. C. Tettamanzi, Nonadiabatic quantum control of valley states in silicon, *Physical Review B* **105**, 075406 (2022).
 - [5] K. Shah, P. Stiller, N. Hoffmann, and A. Cangi, Physics-informed neural networks as solvers for the time-dependent schrödinger equation, *arXiv preprint arXiv:2210.12522* (2022).

Appendix A: Additional Plots

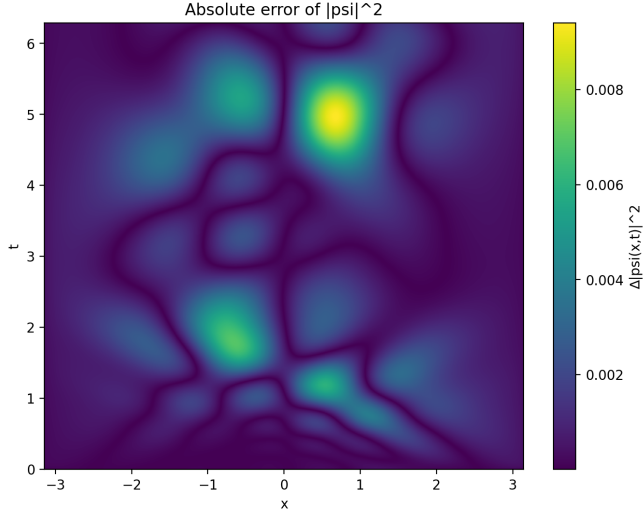


FIG. 10. Absolute error of probability density between the baseline model prediction and analytical solution.

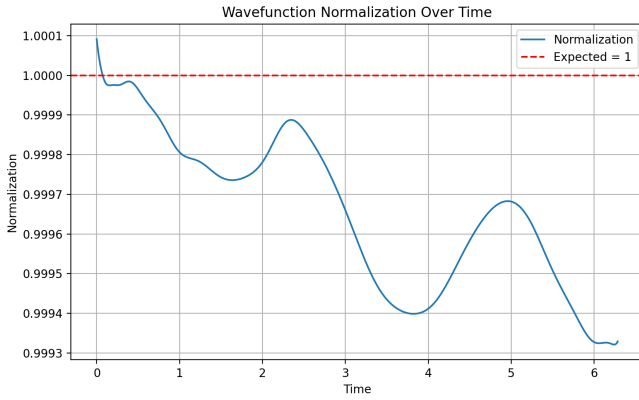


FIG. 11. Wavefunction normalization of the baseline model

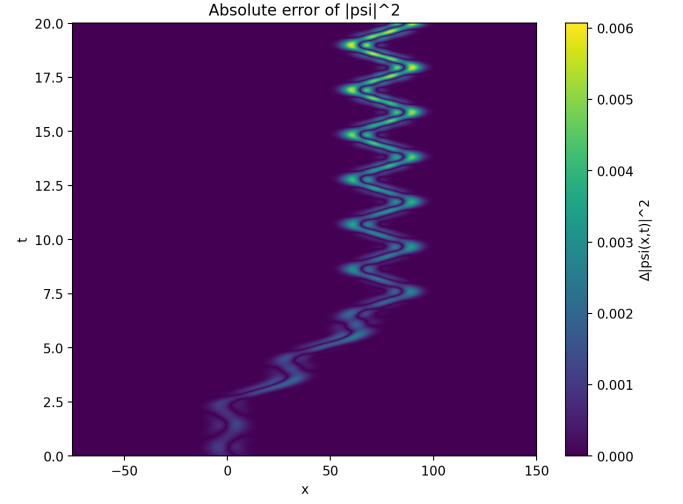


FIG. 12. Absolute error of probability density between the causal model prediction and numerical solution.

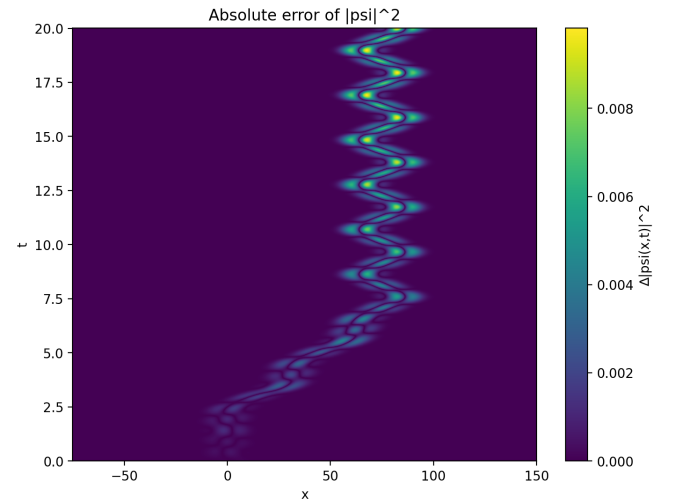


FIG. 13. Absolute error of probability density between the norm model prediction and numerical solution.

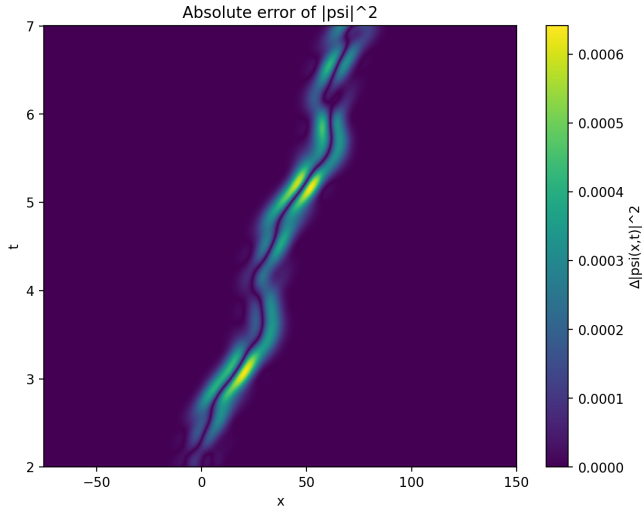


FIG. 14. Absolute error of probability density between the dynamic-window model prediction and numerical solution.

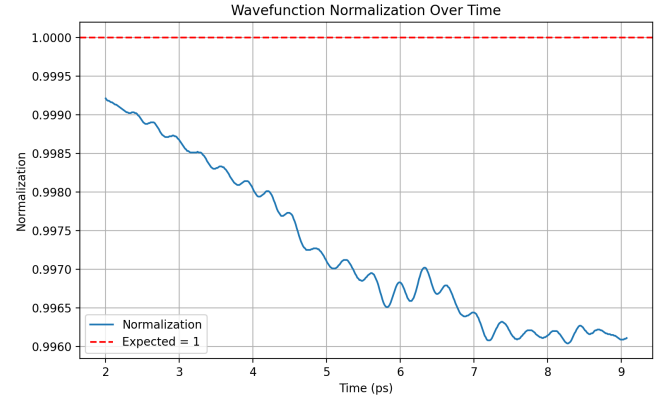


FIG. 16. Wavefunction normalization of the 2-D dynamic-window model

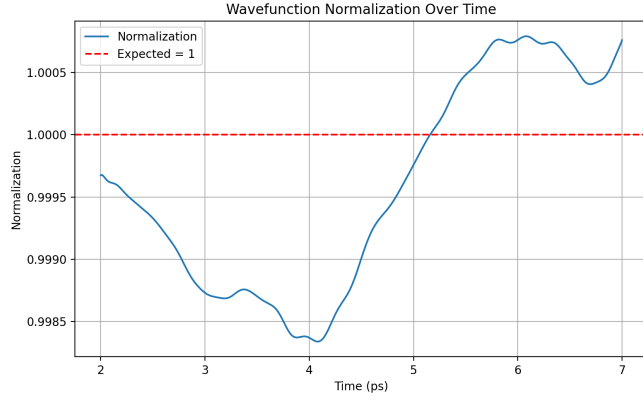


FIG. 15. Wavefunction normalization of the dynamic-window model